

Lógica Digital - Circuitos combinacionales

Segundo Cuatrimestre 2025

Sistemas Digitales
DC - UBA

Algebra de Boole

Partimos de las siguientes proposiciones (axiomas):

Partimos de las siguientes proposiciones (axiomas):

(A1) Existen dos elementos: $X = 1$ si $X \neq 0$ ó $X = 0$ si $X \neq 1$

Partimos de las siguientes proposiciones (axiomas):

(A1) Existen dos elementos: $X = 1$ si $X \neq 0$ ó $X = 0$ si $X \neq 1$

(A2) Existe el operador negación $\overline{()}$ tal que: Si $X = 1 \Rightarrow \overline{X} = 0$

Partimos de las siguientes proposiciones (axiomas):

(A1) Existen dos elementos: $X = 1$ si $X \neq 0$ ó $X = 0$ si $X \neq 1$

(A2) Existe el operador negación $\overline{()}$ tal que: Si $X = 1 \Rightarrow \overline{X} = 0$

(A3) $0 \cdot 0 = 0$ $1 + 1 = 1$

Partimos de las siguientes proposiciones (axiomas):

(A1) Existen dos elementos: $X = 1$ si $X \neq 0$ ó $X = 0$ si $X \neq 1$

(A2) Existe el operador negación $\overline{()}$ tal que: Si $X = 1 \Rightarrow \overline{X} = 0$

(A3) $0 \cdot 0 = 0$ $1 + 1 = 1$

(A4) $1 \cdot 1 = 1$ $0 + 0 = 0$

Partimos de las siguientes proposiciones (axiomas):

(A1) Existen dos elementos: $X = 1$ si $X \neq 0$ ó $X = 0$ si $X \neq 1$

(A2) Existe el operador negación $\overline{()}$ tal que: Si $X = 1 \Rightarrow \overline{X} = 0$

(A3) $0 \cdot 0 = 0$ $1 + 1 = 1$

(A4) $1 \cdot 1 = 1$ $0 + 0 = 0$

(A5) $0 \cdot 1 = 1 \cdot 0 = 0$ $0 + 1 = 1 + 0 = 1$

De los axiomas anteriores se derivan las siguientes propiedades:

Propiedad	AND	OR
Identidad	$1.A = A$	$0 + A = A$

De los axiomas anteriores se derivan las siguientes propiedades:

Propiedad	AND	OR
Identidad	$1.A = A$	$0 + A = A$
Nulo	$0.A = 0$	$1 + A = 1$

De los axiomas anteriores se derivan las siguientes propiedades:

Propiedad	AND	OR
Identidad	$1.A = A$	$0 + A = A$
Nulo	$0.A = 0$	$1 + A = 1$
Idempotencia	$A.A = A$	$A + A = A$

De los axiomas anteriores se derivan las siguientes propiedades:

Propiedad	AND	OR
Identidad	$1.A = A$	$0 + A = A$
Nulo	$0.A = 0$	$1 + A = 1$
Idempotencia	$A.A = A$	$A + A = A$
Inverso	$A.\bar{A} = 0$	$A + \bar{A} = 1$

De los axiomas anteriores se derivan las siguientes propiedades:

Propiedad	AND	OR
Identidad	$1.A = A$	$0 + A = A$
Nulo	$0.A = 0$	$1 + A = 1$
Idempotencia	$A.A = A$	$A + A = A$
Inverso	$A.\bar{A} = 0$	$A + \bar{A} = 1$
Conmutatividad	$A.B = B.A$	$A + B = B + A$

De los axiomas anteriores se derivan las siguientes propiedades:

Propiedad	AND	OR
Identidad	$1.A = A$	$0 + A = A$
Nulo	$0.A = 0$	$1 + A = 1$
Idempotencia	$A.A = A$	$A + A = A$
Inverso	$A.\bar{A} = 0$	$A + \bar{A} = 1$
Conmutatividad	$A.B = B.A$	$A + B = B + A$
Asociatividad	$(A.B).C = A.(B.C)$	$(A + B) + C = A + (B + C)$

De los axiomas anteriores se derivan las siguientes propiedades:

Propiedad	AND	OR
Identidad	$1.A = A$	$0 + A = A$
Nulo	$0.A = 0$	$1 + A = 1$
Idempotencia	$A.A = A$	$A + A = A$
Inverso	$A.\bar{A} = 0$	$A + \bar{A} = 1$
Conmutatividad	$A.B = B.A$	$A + B = B + A$
Asociatividad	$(A.B).C = A.(B.C)$	$(A + B) + C = A + (B + C)$
Distributividad	$A + (B.C) = (A + B).(A + C)$	$A.(B + C) = A.B + A.C$

De los axiomas anteriores se derivan las siguientes propiedades:

Propiedad	AND	OR
Identidad	$1.A = A$	$0 + A = A$
Nulo	$0.A = 0$	$1 + A = 1$
Idempotencia	$A.A = A$	$A + A = A$
Inverso	$A.\bar{A} = 0$	$A + \bar{A} = 1$
Conmutatividad	$A.B = B.A$	$A + B = B + A$
Asociatividad	$(A.B).C = A.(B.C)$	$(A + B) + C = A + (B + C)$
Distributividad	$A + (B.C) = (A + B).(A + C)$	$A.(B + C) = A.B + A.C$
Absorción	$A.(A + B) = A$	$A + A.B = A$

De los axiomas anteriores se derivan las siguientes propiedades:

Propiedad	AND	OR
Identidad	$1.A = A$	$0 + A = A$
Nulo	$0.A = 0$	$1 + A = 1$
Idempotencia	$A.A = A$	$A + A = A$
Inverso	$A.\bar{A} = 0$	$A + \bar{A} = 1$
Conmutatividad	$A.B = B.A$	$A + B = B + A$
Asociatividad	$(A.B).C = A.(B.C)$	$(A + B) + C = A + (B + C)$
Distributividad	$A + (B.C) = (A + B).(A + C)$	$A.(B + C) = A.B + A.C$
Absorción	$A.(A + B) = A$	$A + A.B = A$
De Morgan	$\overline{A.B} = \bar{A} + \bar{B}$	$\overline{A + B} = \bar{A}.\bar{B}$

Tarea: ¡Demostrarlas!

Demostrar si la siguiente igualdad entre funciones booleanas es verdadera o falsa:

$$(X + \bar{Y}) = \overline{(\bar{X} \cdot Y)} \cdot Z + X \cdot \bar{Z} + \overline{(Y + Z)}$$

Demostrar si la siguiente igualdad entre funciones booleanas es verdadera o falsa:

$$(X + \bar{Y}) = \overline{(\bar{X} \cdot Y)} \cdot Z + X \cdot \bar{Z} + \overline{(Y + Z)}$$

Solución:

$$\overline{(\bar{X} \cdot Y)} \cdot Z + X \cdot \bar{Z} + \overline{(Y + Z)} \leftarrow \text{De Morgan}$$

Demostrar si la siguiente igualdad entre funciones booleanas es verdadera o falsa:

$$(X + \bar{Y}) = \overline{(\bar{X} \cdot Y)} \cdot Z + X \cdot \bar{Z} + \overline{(Y + Z)}$$

Solución:

$$\overline{(\bar{X} \cdot Y)} \cdot Z + X \cdot \bar{Z} + \overline{(Y + Z)} \leftarrow \text{De Morgan}$$

$$(\bar{X} \cdot Y) \cdot Z + X \cdot \bar{Z} + \bar{Y} \cdot \bar{Z} \leftarrow \text{Distributiva}$$

Demostrar si la siguiente igualdad entre funciones booleanas es verdadera o falsa:

$$(X + \bar{Y}) = \overline{(\bar{X} \cdot Y)} \cdot Z + X \cdot \bar{Z} + \overline{(Y + Z)}$$

Solución:

$$\overline{(\bar{X} \cdot Y)} \cdot Z + X \cdot \bar{Z} + \overline{(Y + Z)} \leftarrow \text{De Morgan}$$

$$\overline{(\bar{X} \cdot Y)} \cdot Z + X \cdot \bar{Z} + \bar{Y} \cdot \bar{Z} \leftarrow \text{Distributiva}$$

$$\overline{(\bar{X} \cdot Y)} \cdot Z + (X + \bar{Y}) \cdot \bar{Z} \leftarrow \text{De Morgan}$$

Demostrar si la siguiente igualdad entre funciones booleanas es verdadera o falsa:

$$(X + \bar{Y}) = \overline{(\bar{X} \cdot Y)} \cdot Z + X \cdot \bar{Z} + \overline{(Y + Z)}$$

Solución:

$$\overline{(\bar{X} \cdot Y)} \cdot Z + X \cdot \bar{Z} + \overline{(Y + Z)} \leftarrow \text{De Morgan}$$

$$\overline{(\bar{X} \cdot Y)} \cdot Z + X \cdot \bar{Z} + \bar{Y} \cdot \bar{Z} \leftarrow \text{Distributiva}$$

$$\overline{(\bar{X} \cdot Y)} \cdot Z + (X + \bar{Y}) \cdot \bar{Z} \leftarrow \text{De Morgan}$$

$$(X + \bar{Y}) \cdot Z + (X + \bar{Y}) \cdot \bar{Z} \leftarrow \text{Distributiva}$$

Demostrar si la siguiente igualdad entre funciones booleanas es verdadera o falsa:

$$(X + \bar{Y}) = \overline{(\bar{X} \cdot Y)} \cdot Z + X \cdot \bar{Z} + \overline{(Y + Z)}$$

Solución:

$$\overline{(\bar{X} \cdot Y)} \cdot Z + X \cdot \bar{Z} + \overline{(Y + Z)} \leftarrow \text{De Morgan}$$

$$\overline{(\bar{X} \cdot Y)} \cdot Z + X \cdot \bar{Z} + \bar{Y} \cdot \bar{Z} \leftarrow \text{Distributiva}$$

$$\overline{(\bar{X} \cdot Y)} \cdot Z + (X + \bar{Y}) \cdot \bar{Z} \leftarrow \text{De Morgan}$$

$$(X + \bar{Y}) \cdot Z + (X + \bar{Y}) \cdot \bar{Z} \leftarrow \text{Distributiva}$$

$$(X + \bar{Y}) \cdot (Z + \bar{Z}) \leftarrow \text{Inverso}$$

Demostrar si la siguiente igualdad entre funciones booleanas es verdadera o falsa:

$$(X + \bar{Y}) = \overline{(\bar{X} \cdot Y)} \cdot Z + X \cdot \bar{Z} + \overline{(Y + Z)}$$

Solución:

$$\overline{(\bar{X} \cdot Y)} \cdot Z + X \cdot \bar{Z} + \overline{(Y + Z)} \leftarrow \text{De Morgan}$$

$$\overline{(\bar{X} \cdot Y)} \cdot Z + X \cdot \bar{Z} + \bar{Y} \cdot \bar{Z} \leftarrow \text{Distributiva}$$

$$\overline{(\bar{X} \cdot Y)} \cdot Z + (X + \bar{Y}) \cdot \bar{Z} \leftarrow \text{De Morgan}$$

$$(X + \bar{Y}) \cdot Z + (X + \bar{Y}) \cdot \bar{Z} \leftarrow \text{Distributiva}$$

$$(X + \bar{Y}) \cdot (Z + \bar{Z}) \leftarrow \text{Inverso}$$

$$(X + \bar{Y}) \cdot 1 \leftarrow \text{Identidad}$$

Demostrar si la siguiente igualdad entre funciones booleanas es verdadera o falsa:

$$(X + \bar{Y}) = \overline{(\bar{X} \cdot Y)} \cdot Z + X \cdot \bar{Z} + \overline{(Y + Z)}$$

Solución:

$$\overline{(\bar{X} \cdot Y)} \cdot Z + X \cdot \bar{Z} + \overline{(Y + Z)} \leftarrow \text{De Morgan}$$

$$\overline{(\bar{X} \cdot Y)} \cdot Z + X \cdot \bar{Z} + \bar{Y} \cdot \bar{Z} \leftarrow \text{Distributiva}$$

$$\overline{(\bar{X} \cdot Y)} \cdot Z + (X + \bar{Y}) \cdot \bar{Z} \leftarrow \text{De Morgan}$$

$$(X + \bar{Y}) \cdot Z + (X + \bar{Y}) \cdot \bar{Z} \leftarrow \text{Distributiva}$$

$$(X + \bar{Y}) \cdot (Z + \bar{Z}) \leftarrow \text{Inverso}$$

$$(X + \bar{Y}) \cdot 1 \leftarrow \text{Identidad}$$

$X + \bar{Y}$ Lo que queríamos demostrar.

En el lenguaje coloquial vamos a llamar a las operaciones indistintamente de la siguiente forma:

$$A + B \equiv A \text{ OR } B$$

En el lenguaje coloquial vamos a llamar a las operaciones indistintamente de la siguiente forma:

$$A + B \equiv A \text{ OR } B$$

$$AB \equiv A.B \equiv A \text{ AND } B$$

En el lenguaje coloquial vamos a llamar a las operaciones indistintamente de la siguiente forma:

$$A + B \equiv A \text{ OR } B$$

$$AB \equiv A.B \equiv A \text{ AND } B$$

$$\bar{A} \equiv \text{NOT } A$$

Compuertas, señales y tablas de verdad

Son modelos idealizados de dispositivos electrónicos o de computo, que realizan operaciones booleanas.

Las podemos representar gráficamente:



Son modelos idealizados de dispositivos electrónicos o de computo, que realizan operaciones booleanas.

Las podemos representar gráficamente:



O describir mediante un lenguaje de descripción de hardware (HDL), por ejemplo en SystemVerilog:

```
assign o = a & b;
```

Son representaciones que nos permiten observar todas las salidas para todas las combinaciones de entradas¹.

Por ejemplo, la función del ejercicio ($F = X + \overline{Y}$) se representa:

X	Y	F
0	0	
0	1	
1	0	
1	1	

¹ Como resulta esperable, esta representación puede volverse muy compleja cuando el número de variables y salidas crece.

Son representaciones que nos permiten observar todas las salidas para todas las combinaciones de entradas¹.

Por ejemplo, la función del ejercicio ($F = X + \overline{Y}$) se representa:

X	Y	F
0	0	1
0	1	
1	0	
1	1	

¹ Como resulta esperable, esta representación puede volverse muy compleja cuando el número de variables y salidas crece.

Son representaciones que nos permiten observar todas las salidas para todas las combinaciones de entradas¹.

Por ejemplo, la función del ejercicio ($F = X + \bar{Y}$) se representa:

X	Y	F
0	0	1
0	1	0
1	0	
1	1	

¹ Como resulta esperable, esta representación puede volverse muy compleja cuando el número de variables y salidas crece.

Son representaciones que nos permiten observar todas las salidas para todas las combinaciones de entradas¹.

Por ejemplo, la función del ejercicio ($F = X + \bar{Y}$) se representa:

X	Y	F
0	0	1
0	1	0
1	0	1
1	1	

¹ Como resulta esperable, esta representación puede volverse muy compleja cuando el número de variables y salidas crece.

Son representaciones que nos permiten observar todas las salidas para todas las combinaciones de entradas¹.

Por ejemplo, la función del ejercicio ($F = X + \bar{Y}$) se representa:

X	Y	F
0	0	1
0	1	0
1	0	1
1	1	1

¹ Como resulta esperable, esta representación puede volverse muy compleja cuando el número de variables y salidas crece.

Gráficamente:



Tabla de verdad:

A	NOT A
0	1
1	0

En SystemVerilog:

```
assign o = ~a;
```

Gráficamente:



Tabla de verdad:

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

En SystemVerilog:

```
assign o = a & b;
```

Gráficamente:



Tabla de verdad:

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

En SystemVerilog:

```
assign o = a | b;
```

Gráficamente:



Tabla de verdad:

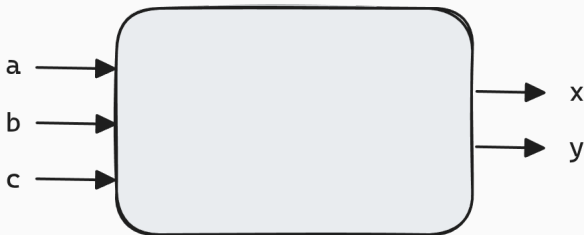
A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

En SystemVerilog:

```
assign o = a ^ b;
```

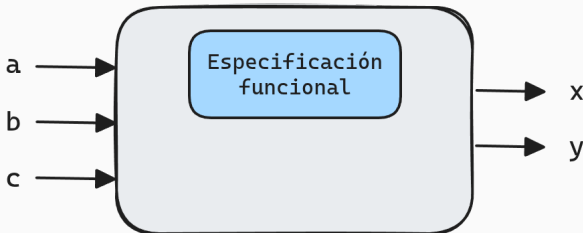
Un **sistema digital** es una red que procesa variables con valores discretos.

Un sistema puede ser visto como una *caja negra*:



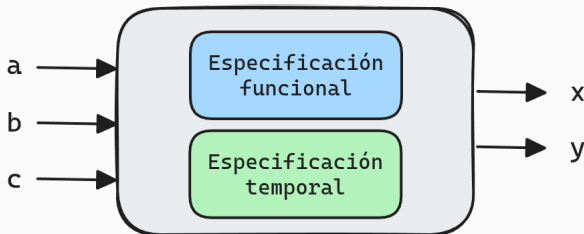
Un **sistema digital** es una red que procesa variables con valores discretos.

Un sistema puede ser visto como una *caja negra*:

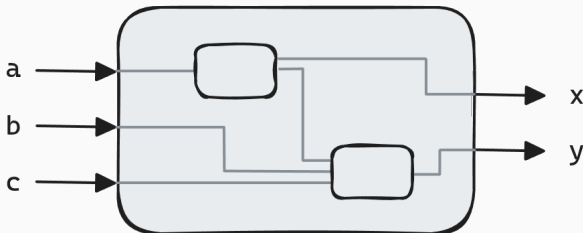


Un **sistema digital** es una red que procesa variables con valores discretos.

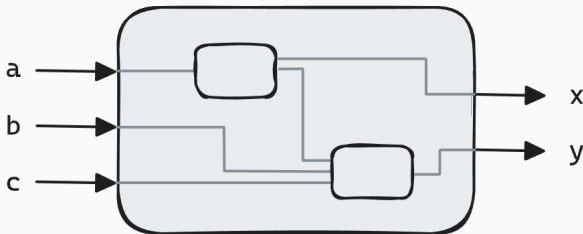
Un sistema puede ser visto como una *caja negra*:



A su vez, un sistema puede estar compuesto por subsistemas, que a su vez pueden estar compuestos por otros subsistemas, y así sucesivamente.



A su vez, un sistema puede estar compuesto por subsistemas, que a su vez pueden estar compuestos por otros subsistemas, y así sucesivamente.



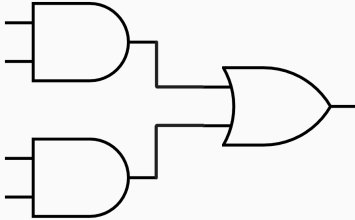
Generalmente, usamos el nivel de **abstracción** que más nos conviene según el problema que estemos resolviendo.

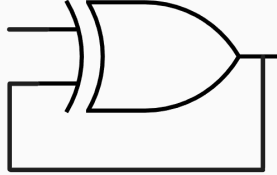
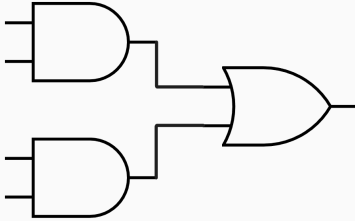
Un sistema digital puede ser:

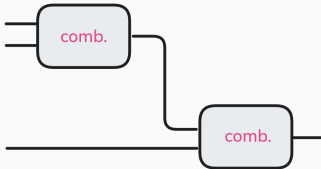
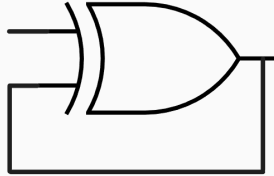
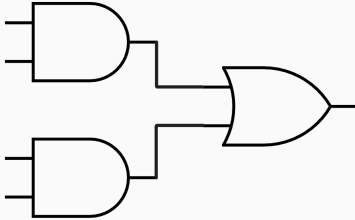
- **Combinacional:** si sus salidas dependen únicamente de las entradas actuales.

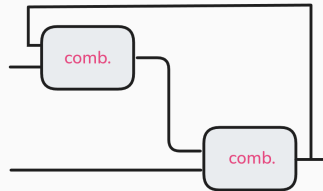
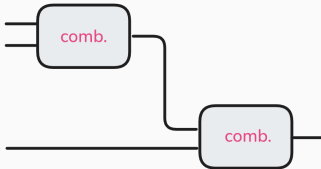
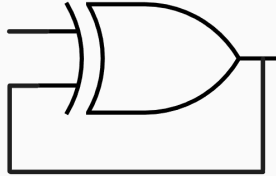
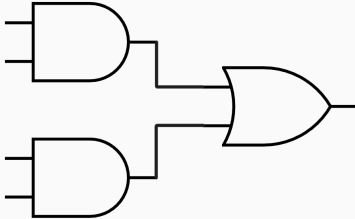
Un sistema digital puede ser:

- **Combinacional:** si sus salidas dependen únicamente de las entradas actuales.
- **Secuencial:** si sus salidas dependen de las entradas actuales y de entradas anteriores (tiene memoria).





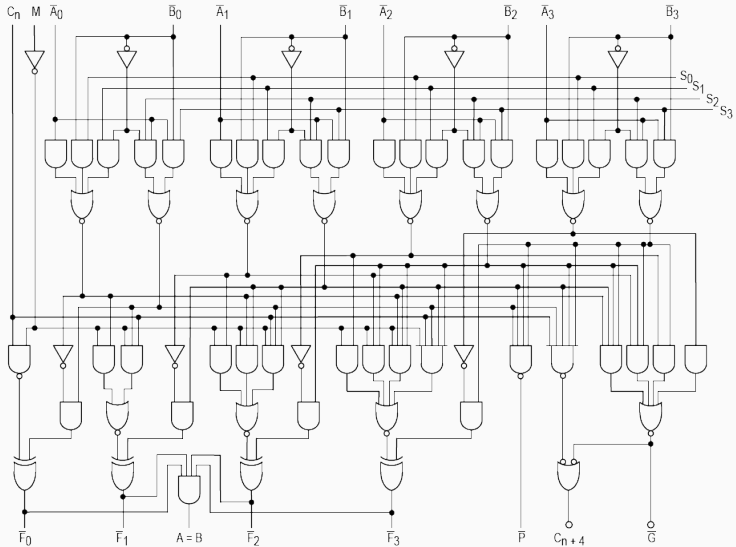




Entradas y salidas - Categorización

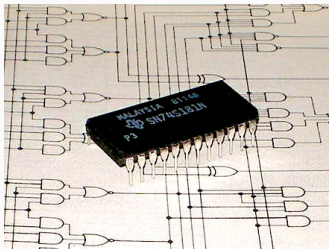
Por momentos vamos a querer abstraer nuestros circuitos en módulos de los cuales observaremos solamente sus entradas y salidas.

Veamos un ejemplo donde ocultamos parte de la complejidad pasando de una vista interna del circuito (caja blanca) a una externa (caja negra).

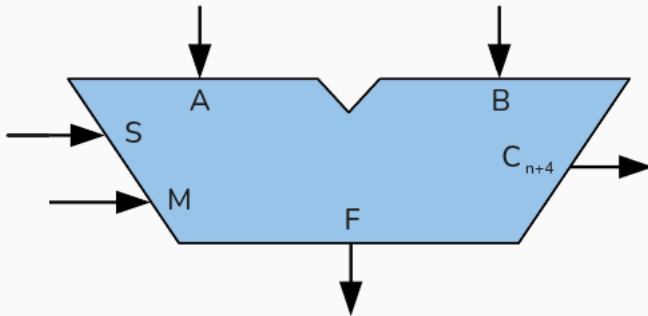


- La primer ALU completa de 4 bits integrada en un solo chip
- 16 funciones lógicas y 16 funciones aritméticas
- Podía realizar todas las operaciones tradicionales: suma, resta, decremento, con o sin carry, así como AND, NAND, OR, NOR, XOR y shift, entre otras.

- La primer ALU completa de 4 bits integrada en un solo chip
- 16 funciones lógicas y 16 funciones aritméticas
- Podía realizar todas las operaciones tradicionales: suma, resta, decremento, con o sin carry, así como AND, NAND, OR, NOR, XOR y shift, entre otras.



Aplicando lo anterior, podemos trabajar con la ALU viéndola de la siguiente manera:



Establecen el sentido de la información:

En la ALU anterior se representan con las flechas...

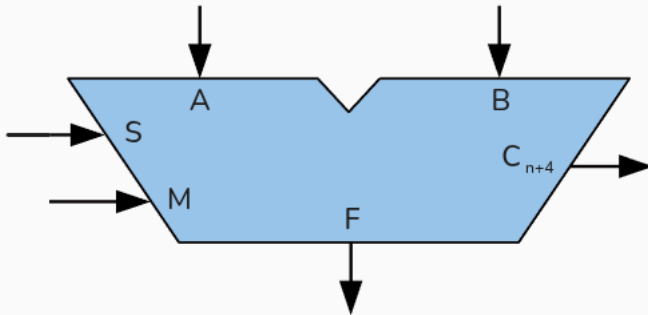
Establecen el sentido de la información:

En la ALU anterior se representan con las flechas...

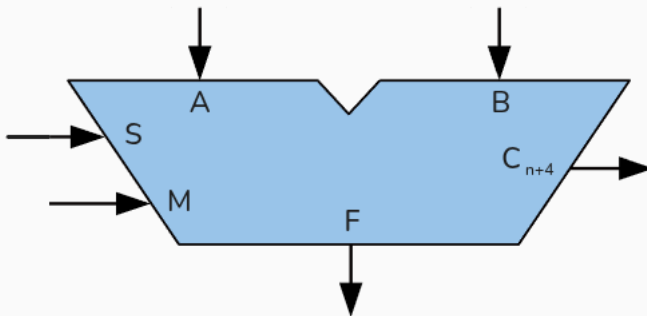
En SystemVerilog:

```
module ALU #(parameter WIDTH = 4)
(
    input [WIDTH-1:0] A,
    input [WIDTH-1:0] B,
    input [3:0] op,
    output [WIDTH-1:0] result,
    output carry_out
);
end module;
```

En la ALU, ¿son funcionalmente todas iguales las entradas y salidas?



En la ALU, ¿son funcionalmente todas iguales las entradas y salidas?



NO

Datos vs. Control

Lógica proposicional a circuitos combinacionales

El estudio de la lógica proposicional y del álgebra de Boole tiene que ver con que vamos a querer implementar funciones lógicas utilizando sistemas combinacionales.

La primera forma que vamos a ver es la llamada **suma de productos** (o *Sum of Products*, **SOP**).

Supongamos que queremos implementar la función lógica $Y = f(A, B)$ tal que:

La primera forma que vamos a ver es la llamada **suma de productos** (o *Sum of Products*, **SOP**).

Supongamos que queremos implementar la función lógica $Y = f(A, B)$ tal que:

A	B	Y		
0	0	0		
0	1	1		
1	0	0		
1	1	0		

La primera forma que vamos a ver es la llamada **suma de productos** (o *Sum of Products*, **SOP**).

Supongamos que queremos implementar la función lógica $Y = f(A, B)$ tal que:

A	B	Y	minitérmino	nombre
0	0	0	$\overline{A} \cdot \overline{B}$	m_0
0	1	1	$\overline{A} \cdot B$	m_1
1	0	0	$A \cdot \overline{B}$	m_2
1	1	0	$A \cdot B$	m_3

La primera forma que vamos a ver es la llamada **suma de productos** (o *Sum of Products*, **SOP**).

Supongamos que queremos implementar la función lógica $Y = f(A, B)$ tal que:

A	B	Y	minitérmino	nombre
0	0	0	$\bar{A} \cdot \bar{B}$	m_0
0	1	1	$\bar{A} \cdot B$	m_1
1	0	0	$A \cdot \bar{B}$	m_2
1	1	0	$A \cdot B$	m_3

La primera forma que vamos a ver es la llamada **suma de productos** (o *Sum of Products*, **SOP**).

Supongamos que queremos implementar la función lógica $Y = f(A, B)$ tal que:

A	B	Y	minitérmino	nombre
0	0	0	$\bar{A} \cdot \bar{B}$	m_0
0	1	1	$\bar{A} \cdot B$	m_1
1	0	0	$A \cdot \bar{B}$	m_2
1	1	0	$A \cdot B$	m_3

Entonces nos queda:

$$Y = f(A, B) = \bar{A} \cdot B = m_1$$

Y si tenemos más de un 1 en la salida de la tabla de verdad, por ejemplo:

A	B	Y		
0	0	0		
0	1	1		
1	0	0		
1	1	1		

Y si tenemos más de un 1 en la salida de la tabla de verdad, por ejemplo:

A	B	Y	minitérmino	nombre
0	0	0	$\overline{A} \cdot \overline{B}$	m_0
0	1	1	$\overline{A} \cdot B$	m_1
1	0	0	$A \cdot \overline{B}$	m_2
1	1	1	$A \cdot B$	m_3

Y si tenemos más de un 1 en la salida de la tabla de verdad, por ejemplo:

A	B	Y	minitérmino	nombre
0	0	0	$\bar{A} \cdot \bar{B}$	m_0
0	1	1	$\bar{A} \cdot B$	m_1
1	0	0	$A \cdot \bar{B}$	m_2
1	1	1	$A \cdot B$	m_3

Y si tenemos más de un 1 en la salida de la tabla de verdad, por ejemplo:

A	B	Y	minitérmino	nombre
0	0	0	$\bar{A} \cdot \bar{B}$	m_0
0	1	1	$\bar{A} \cdot B$	m_1
1	0	0	$A \cdot \bar{B}$	m_2
1	1	1	$A \cdot B$	m_3

$$Y = f(A, B) = \bar{A} \cdot B + A \cdot B = m_1 + m_3$$

Y si tenemos más de un 1 en la salida de la tabla de verdad, por ejemplo:

A	B	Y	minitérmino	nombre
0	0	0	$\bar{A} \cdot \bar{B}$	m_0
0	1	1	$\bar{A} \cdot B$	m_1
1	0	0	$A \cdot \bar{B}$	m_2
1	1	1	$A \cdot B$	m_3

$$Y = f(A, B) = \bar{A} \cdot B + A \cdot B = m_1 + m_3$$

$$Y = \sum m(1, 3)$$

Y si tenemos más de 2 variables:

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

Y si tenemos más de 2 variables:

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

Y si tenemos más de 2 variables:

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

$$Y = f(A, B, C) = \bar{A} \cdot \bar{B} \cdot \bar{C} + A \cdot \bar{B} \cdot \bar{C} + A \cdot \bar{B} \cdot C = m_0 + m_4 + m_5$$

Y si tenemos más de 2 variables:

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

$$Y = f(A, B, C) = \bar{A} \cdot \bar{B} \cdot \bar{C} + A \cdot \bar{B} \cdot \bar{C} + A \cdot \bar{B} \cdot C = m_0 + m_4 + m_5$$

$$Y = \sum m(0, 4, 5)$$

La segunda forma que vamos a ver es la llamada **producto de sumas (POS)**.

La segunda forma que vamos a ver es la llamada **producto de sumas (POS)**.

A	B	Y		
0	0	0		
0	1	1		
1	0	0		
1	1	0		

La segunda forma que vamos a ver es la llamada **producto de sumas (POS)**.

A	B	Y	maxitérmino	nombre
0	0	0	$A + B$	M_0
0	1	1	$A + \overline{B}$	M_1
1	0	0	$\overline{A} + B$	M_2
1	1	0	$\overline{A} + \overline{B}$	M_3

La segunda forma que vamos a ver es la llamada **producto de sumas (POS)**.

A	B	Y	maxitérmino	nombre
0	0	0	$A + B$	M_0
0	1	1	$A + \bar{B}$	M_1
1	0	0	$\bar{A} + B$	M_2
1	1	0	$\bar{A} + \bar{B}$	M_3

La segunda forma que vamos a ver es la llamada **producto de sumas (POS)**.

A	B	Y	maxitérmino	nombre
0	0	0	$A + B$	M_0
0	1	1	$A + \bar{B}$	M_1
1	0	0	$\bar{A} + B$	M_2
1	1	0	$\bar{A} + \bar{B}$	M_3

$$Y = f(A, B) = (A + B) \cdot (\bar{A} + B) \cdot (\bar{A} + \bar{B}) = M_0 \cdot M_2 \cdot M_3$$

La segunda forma que vamos a ver es la llamada **producto de sumas (POS)**.

A	B	Y	maxitérmino	nombre
0	0	0	$A + B$	M_0
0	1	1	$A + \bar{B}$	M_1
1	0	0	$\bar{A} + B$	M_2
1	1	0	$\bar{A} + \bar{B}$	M_3

$$Y = f(A, B) = (A + B) \cdot (\bar{A} + B) \cdot (\bar{A} + \bar{B}) = M_0 \cdot M_2 \cdot M_3$$

$$Y = \prod M(0, 2, 3)$$

Circuitos básicos

Suma dos bits y produce la suma y el acarreo.

Sumador simple (*half-adder*)

Suma dos bits y produce la suma y el acarreo.

Tabla de verdad:

a_i	b_i	s_o	c_o
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Sumador simple (*half-adder*)

Suma dos bits y produce la suma y el acarreo.

Tabla de verdad:

a_i	b_i	s_o	c_o
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Ecuaciones lógicas:

$$s_o = (a_i \cdot \overline{b_i}) + (\overline{a_i} \cdot b_i)$$

$$= a_i \oplus b_i$$

$$c_o = a_i \cdot b_i$$

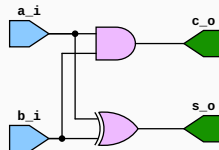
Sumador simple (*half-adder*)

Suma dos bits y produce la suma y el acarreo.

Tabla de verdad:

a_i	b_i	s_o	c_o
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Implementación:



Ecuaciones lógicas:

$$s_o = (a_i \cdot \overline{b_i}) + (\overline{a_i} \cdot b_i)$$

$$= a_i \oplus b_i$$

$$c_o = a_i \cdot b_i$$

Sumador simple (*half-adder*)

Suma dos bits y produce la suma y el acarreo.

Tabla de verdad:

a_i	b_i	s_o	c_o
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

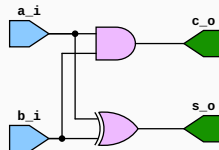
Ecuaciones lógicas:

$$s_o = (a_i \cdot \overline{b_i}) + (\overline{a_i} \cdot b_i)$$

$$= a_i \oplus b_i$$

$$c_o = a_i \cdot b_i$$

Implementación:



```
module half_adder (  
    input a_i,  
    input b_i,  
    output s_o,  
    output c_o  
);  
  
    assign s_o = a_i ^ b_i;  
    assign c_o = a_i & b_i;  
  
endmodule
```

Suma dos bits y un acarreo de entrada, y produce la suma y el acarreo de salida.

Suma dos bits y un acarreo de entrada, y produce la suma y el acarreo de salida.

Tabla de verdad:

c_i	a_i	b_i	s_o	c_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Sumador completo (*full-adder*)

Suma dos bits y un acarreo de entrada, y produce la suma y el acarreo de salida.

Tabla de verdad:

c_i	a_i	b_i	s_o	c_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Ecuaciones lógicas:

Sumador completo (*full-adder*)

Suma dos bits y un acarreo de entrada, y produce la suma y el acarreo de salida.

Tabla de verdad:

c_i	a_i	b_i	s_o	c_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Ecuaciones lógicas:

$$\begin{aligned}s_o &= (\overline{c_i} \cdot \overline{a_i} \cdot b_i) + (\overline{c_i} \cdot a_i \cdot \overline{b_i}) \\ &\quad + (c_i \cdot \overline{a_i} \cdot \overline{b_i}) + (c_i \cdot a_i \cdot b_i) \\ &= (a_i \oplus b_i) \oplus C_{in}\end{aligned}$$

$$\begin{aligned}c_o &= (\overline{c_i} \cdot a_i \cdot b_i) + (c_i \cdot \overline{a_i} \cdot b_i) \\ &\quad + (c_i \cdot a_i \cdot \overline{b_i}) + (c_i \cdot a_i \cdot b_i) \\ &= (a_i \cdot b_i) + (c_i \cdot (a_i \oplus b_i))\end{aligned}$$

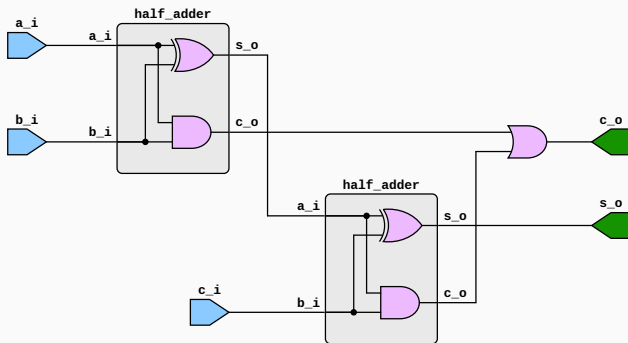
Ya se pone complicado...

Usemos lógica composicional...

Sumador completo (*full-adder*)

Usemos lógica composicional...

Ya tenemos el sumador simple, entonces:



O en SystemVerilog:

```
// Full adder using two half adders
```

```
`include "half_adder.sv"
```

```
module full_adder (
```

```
    input a_i,
```

```
    input b_i,
```

```
    input c_i,
```

```
    output s_o,
```

```
    output c_o
```

```
);
```

```
    wire s1, c1, c2;
```

```
    assign c_o = c1 | c2;
```

(... continúa al lado ...)

```
    half_adder ha1 (
```

```
        .a_i(a_i),
```

```
        .b_i(b_i),
```

```
        .s_o(s1),
```

```
        .c_o(c1)
```

```
    );
```

```
    half_adder ha2 (
```

```
        .a_i(s1),
```

```
        .b_i(c_i),
```

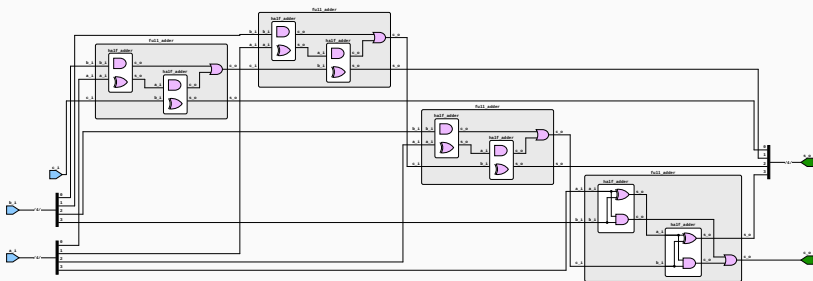
```
        .s_o(s_o),
```

```
        .c_o(c2)
```

```
    );
```

```
endmodule
```

Ahora podemos escalar... :D



O en SystemVerilog:

```
`include "full_adder.sv"

module ripple_carry_adder
// Default to a 4-bit adder
#(parameter N=4) (
    input [N-1:0] a_i,
    input [N-1:0] b_i,
    input c_i,
    output [N-1:0] s_o,
    output c_o

);

// N+1 carry bits
wire [N:0] c_internal;

assign c_internal[0] = c_i;

    genvar i;
    generate
        for (i = 0; i < N; i = i + 1)
            begin : fa_gen
                full_adder fa_inst (
                    .a_i(a_i[i]),
                    .b_i(b_i[i]),
                    .c_i(c_internal[i]),
                    .s_o(s_o[i]),
                    .c_o(c_internal[i+1])
                );
            end
    endgenerate

    assign c_o = c_internal[N];

endmodule
```

(... continúa al lado ...)

Armar un circuito de 3 *bits*. Este deberá mover a izquierda o a derecha los bits de entrada de acuerdo al valor de una entrada extra que actúa como control. En otras palabras, un shift *izq-der* de k -bits es un circuito de $k + 1$ entradas ($e_k, \dots, e_0$) y k salidas ($s_{k-1}, \dots, s_0$) que funciona del siguiente modo:

- Si $e_k = 1$, entonces $s_i = e_{i-1}$ para todo $0 < i < k$ y $s_0 = 0$
- Si $e_k = 0$, entonces $s_i = e_{i+1}$ para todo $0 \leq i < k - 1$ y $s_{k-1} = 0$

Armar un circuito de 3 *bits*. Este deberá mover a izquierda o a derecha los bits de entrada de acuerdo al valor de una entrada extra que actúa como control. En otras palabras, un shift *izq-der* de k -bits es un circuito de $k + 1$ entradas ($e_k, \dots, e_0$) y k salidas ($s_{k-1}, \dots, s_0$) que funciona del siguiente modo:

- Si $e_k = 1$, entonces $s_i = e_{i-1}$ para todo $0 < i < k$ y $s_0 = 0$
- Si $e_k = 0$, entonces $s_i = e_{i+1}$ para todo $0 \leq i < k - 1$ y $s_{k-1} = 0$

Ejemplos:

$$\text{shift_lr}(1, 011) = 110$$

Armar un circuito de 3 *bits*. Este deberá mover a izquierda o a derecha los bits de entrada de acuerdo al valor de una entrada extra que actúa como control. En otras palabras, un shift *izq-der* de k -bits es un circuito de $k + 1$ entradas ($e_k, \dots, e_0$) y k salidas ($s_{k-1}, \dots, s_0$) que funciona del siguiente modo:

- Si $e_k = 1$, entonces $s_i = e_{i-1}$ para todo $0 < i < k$ y $s_0 = 0$
- Si $e_k = 0$, entonces $s_i = e_{i+1}$ para todo $0 \leq i < k - 1$ y $s_{k-1} = 0$

Ejemplos:

$$\text{shift_lr}(1,011) = 110 \quad \text{shift_lr}(0,011) = 001$$

Armar un circuito de 3 *bits*. Este deberá mover a izquierda o a derecha los bits de entrada de acuerdo al valor de una entrada extra que actúa como control. En otras palabras, un *shift izq-der* de k -bits es un circuito de $k + 1$ entradas ($e_k, \dots, e_0$) y k salidas ($s_{k-1}, \dots, s_0$) que funciona del siguiente modo:

- Si $e_k = 1$, entonces $s_i = e_{i-1}$ para todo $0 < i < k$ y $s_0 = 0$
- Si $e_k = 0$, entonces $s_i = e_{i+1}$ para todo $0 \leq i < k - 1$ y $s_{k-1} = 0$

Ejemplos:

$$\begin{aligned}\text{shift_lr}(1,011) &= 110 & \text{shift_lr}(0,011) &= 001 \\ \text{shift_lr}(1,100) &= 000\end{aligned}$$

Armar un circuito de 3 *bits*. Este deberá mover a izquierda o a derecha los bits de entrada de acuerdo al valor de una entrada extra que actúa como control. En otras palabras, un *shift izq-der* de k -bits es un circuito de $k + 1$ entradas ($e_k, \dots, e_0$) y k salidas ($s_{k-1}, \dots, s_0$) que funciona del siguiente modo:

- Si $e_k = 1$, entonces $s_i = e_{i-1}$ para todo $0 < i < k$ y $s_0 = 0$
- Si $e_k = 0$, entonces $s_i = e_{i+1}$ para todo $0 \leq i < k - 1$ y $s_{k-1} = 0$

Ejemplos:

$$\begin{aligned} \text{shift_lr}(1, 011) &= 110 & \text{shift_lr}(0, 011) &= 001 \\ \text{shift_lr}(1, 100) &= 000 & \text{shift_lr}(1, 101) &= 010 \end{aligned}$$

- Si $e_k = 1$, entonces $s_i = e_{i-1}$ para todo $0 < i < k$ y $s_0 = 0$
- Si $e_k = 0$, entonces $s_i = e_{i+1}$ para todo $0 \leq i < k - 1$ y $s_{k-1} = 0$

Solución:

- Si $e_k = 1$, entonces $s_i = e_{i-1}$ para todo $0 < i < k$ y $s_0 = 0$
- Si $e_k = 0$, entonces $s_i = e_{i+1}$ para todo $0 \leq i < k - 1$ y $s_{k-1} = 0$

Solución:

$$s_2 = \begin{bmatrix} 0 & \text{si } e_3 = 0 \\ e_1 & \text{si } e_3 = 1 \end{bmatrix}$$

- Si $e_k = 1$, entonces $s_i = e_{i-1}$ para todo $0 < i < k$ y $s_0 = 0$
- Si $e_k = 0$, entonces $s_i = e_{i+1}$ para todo $0 \leq i < k - 1$ y $s_{k-1} = 0$

Solución:

$$s_2 = \begin{bmatrix} 0 & \text{si } e_3 = 0 \\ e_1 & \text{si } e_3 = 1 \end{bmatrix}$$

$$e_3.e_1$$

- Si $e_k = 1$, entonces $s_i = e_{i-1}$ para todo $0 < i < k$ y $s_0 = 0$
- Si $e_k = 0$, entonces $s_i = e_{i+1}$ para todo $0 \leq i < k - 1$ y $s_{k-1} = 0$

Solución:

$$s_2 = \begin{bmatrix} 0 & \text{si } e_3 = 0 \\ e_1 & \text{si } e_3 = 1 \end{bmatrix} \quad s_0 = \begin{bmatrix} 0 & \text{si } e_3 = 1 \\ e_1 & \text{si } e_3 = 0 \end{bmatrix}$$

$$e_3.e_1$$

- Si $e_k = 1$, entonces $s_i = e_{i-1}$ para todo $0 < i < k$ y $s_0 = 0$
- Si $e_k = 0$, entonces $s_i = e_{i+1}$ para todo $0 \leq i < k - 1$ y $s_{k-1} = 0$

Solución:

$$s_2 = \begin{bmatrix} 0 & \text{si } e_3 = 0 \\ e_1 & \text{si } e_3 = 1 \end{bmatrix} \quad s_0 = \begin{bmatrix} 0 & \text{si } e_3 = 1 \\ e_1 & \text{si } e_3 = 0 \end{bmatrix}$$

$$e_3.e_1$$

$$\overline{e_3}.e_1$$

- Si $e_k = 1$, entonces $s_i = e_{i-1}$ para todo $0 < i < k$ y $s_0 = 0$
- Si $e_k = 0$, entonces $s_i = e_{i+1}$ para todo $0 \leq i < k - 1$ y $s_{k-1} = 0$

Solución:

$$s_2 = \begin{bmatrix} 0 & \text{si } e_3 = 0 \\ e_1 & \text{si } e_3 = 1 \end{bmatrix} \quad s_0 = \begin{bmatrix} 0 & \text{si } e_3 = 1 \\ e_1 & \text{si } e_3 = 0 \end{bmatrix} \quad s_1 = \begin{bmatrix} e_0 & \text{si } e_3 = 1 \\ e_2 & \text{si } e_3 = 0 \end{bmatrix}$$

$$e_3.e_1$$

$$\overline{e_3}.e_1$$

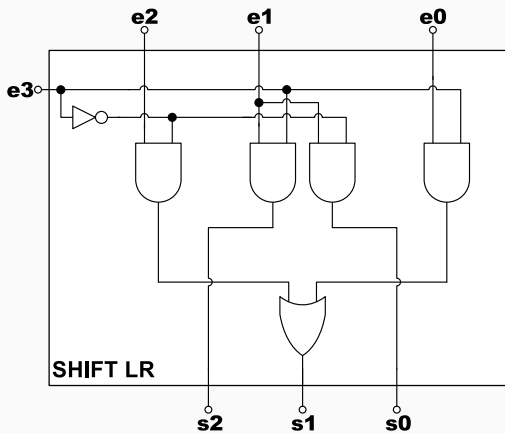
- Si $e_k = 1$, entonces $s_i = e_{i-1}$ para todo $0 < i < k$ y $s_0 = 0$
- Si $e_k = 0$, entonces $s_i = e_{i+1}$ para todo $0 \leq i < k - 1$ y $s_{k-1} = 0$

Solución:

$$s_2 = \begin{bmatrix} 0 & \text{si } e_3 = 0 \\ e_1 & \text{si } e_3 = 1 \end{bmatrix} \quad s_0 = \begin{bmatrix} 0 & \text{si } e_3 = 1 \\ e_1 & \text{si } e_3 = 0 \end{bmatrix} \quad s_1 = \begin{bmatrix} e_0 & \text{si } e_3 = 1 \\ e_2 & \text{si } e_3 = 0 \end{bmatrix}$$

$e_3.e_1$ $\overline{e_3}.e_1$ $e_3.e_0 + \overline{e_3}.e_2$

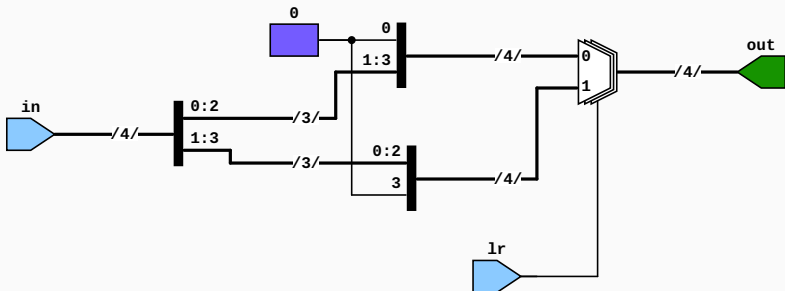
Solución:



En HDL (SystemVerilog en este caso) podemos trabajar en diferentes niveles de abstracción.

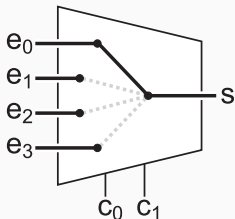
```
module shiftlr #(parameter N = 4) (  
    input  logic [N-1:0] in,  
    input  logic          lr,  
    output logic [N-1:0] out  
);  
  
    always_comb begin  
        if (lr == 1'b0) begin  
            out = {in[N-2:0], 1'b0};  
        end else begin  
            out = {1'b0, in[N-1:1]};  
        end  
    end  
end  
  
endmodule
```

Al **elaborar** el código, la herramienta **yosys**² genera el siguiente diagrama estructural:



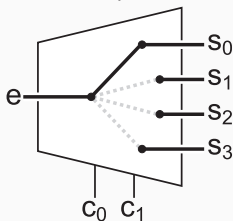
²<https://yosyshq.net/yosys/>

Multiplexor

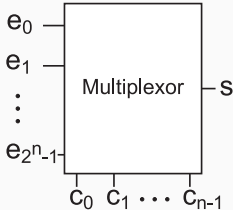


Las líneas de control c permiten seleccionar una de las entradas e , la que corresponderá a la salida s .

Demultiplexor

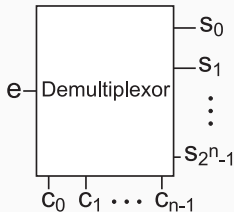


Las líneas de control c permiten seleccionar cual de las salidas s tendrá el valor de e .



Mux:

- 2^n entradas.
- n líneas de control.
- 1 salida.
- Las entradas/salidas pueden tener N bits (tamaño de palabra).



Demux:

- 1 entrada.
- n líneas de control.
- 2^n salidas.
- Las entradas/salidas pueden tener N bits (tamaño de palabra).

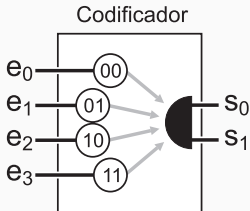
En SystemVerilog:

Sintaxis vieja:

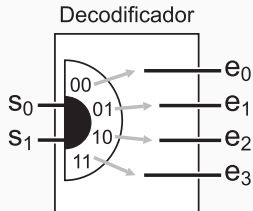
```
module mux #(parameter N = 4,  
              parameter M = 4) (  
    input  logic [N*M-1:0] in,  
    input  logic [$clog2(M)-1:0] sel,  
    output logic [N-1:0] out  
);  
  
always_comb begin  
    out = in[sel*N +: N];  
end  
  
endmodule
```

Sintaxis nueva:

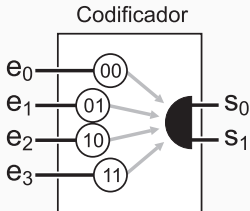
```
module mux #(parameter N = 4,  
              parameter M = 4) (  
    input logic [N-1:0] in [M-1:0],  
    input logic [$clog2(M)-1:0] sel,  
    output logic [N-1:0] out  
);  
  
always_comb begin  
    out = in[sel];  
end  
  
endmodule
```



Una y sólo una línea en alto de e corresponderá a una combinación en la salida s .

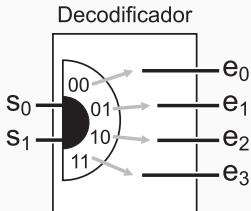


Cada combinación de las líneas e corresponderá a una sola línea en alto de la salida s .

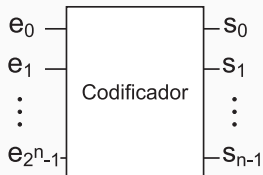


Una y sólo una línea en alto de e corresponderá a una combinación en la salida s .

Si más de una línea en e está en alto, el valor de la salida s es indeterminado.

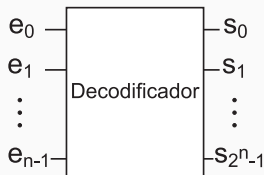


Cada combinación de las líneas e corresponderá a una sola línea en alto de la salida s .



Codificador:

- 2^n entradas.
- n salidas.



Decodificador:

- n entradas.
- 2^n salidas.

En SystemVerilog:

```
module encoder #(parameter N = 4) (  
    input  logic [N-1:0] in,  
    output logic [$clog2(N)-1:0] out  
);  
  
    always_comb begin  
        out = 0;  
        for (int i = 0; i < N; i++)  
            begin  
                if (in[i]) begin  
                    out = i;  
                    // OJO! ¿Qué pasa acá?  
                end  
            end  
        end  
    end  
endmodule
```

Y si hay más de una entrada activa...

¿Cuál es la salida esperada?

¿Qué genera este código?

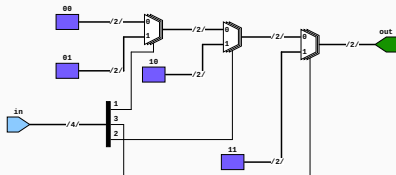
En SystemVerilog:

```
module encoder #(parameter N = 4) (
    input  logic [N-1:0] in,
    output logic [$clog2(N)-1:0] out
);
    always_comb begin
        out = 0;
        for (int i = 0; i < N; i++)
            begin
                if (in[i]) begin
                    out = i;
                    // OJO! ¿Qué pasa acá?
                end
            end
        end
    end
endmodule
```

Y si hay más de una entrada activa...

¿Cuál es la salida esperada?

¿Qué genera este código?
Depende...



Timing



¡Las compuertas no son instantáneas!

¡Las compuertas no son instantáneas!

- **Tiempo de propagación:** tiempo máximo hasta que la salida se estabiliza

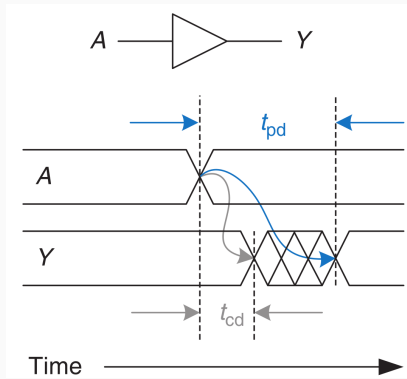
¡Las compuertas no son instantáneas!

- **Tiempo de propagación:** tiempo máximo hasta que la salida se estabiliza
- **Tiempo de contaminación:** tiempo mínimo hasta que la salida comienza a cambiar (a.k.a. tiempo de transición de salida)

¡Las compuertas no son instantáneas!

- **Tiempo de propagación:** tiempo máximo hasta que la salida se estabiliza
- **Tiempo de contaminación:** tiempo mínimo hasta que la salida comienza a cambiar (a.k.a. tiempo de transición de salida)

Visualmente:

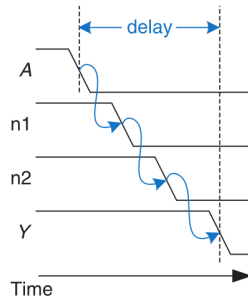
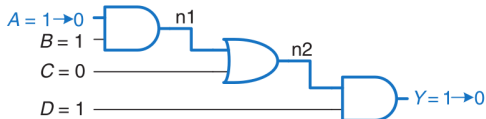




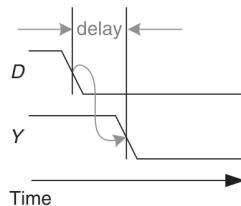
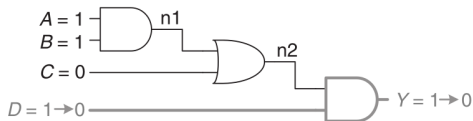
¿Cómo impacta esto en los sistemas que diseñamos?

¿Cómo impacta esto en los sistemas que diseñamos?

Critical Path



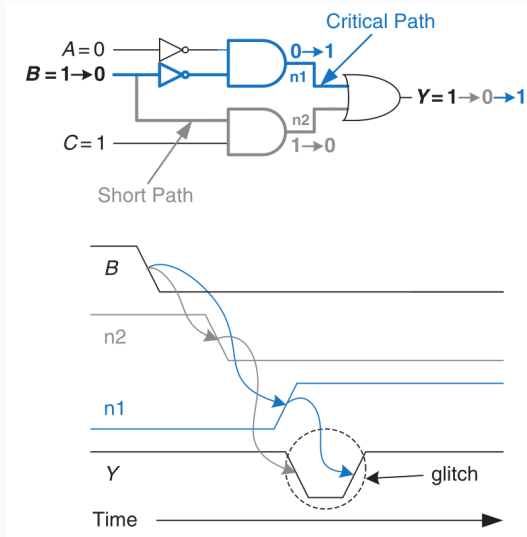
Short Path



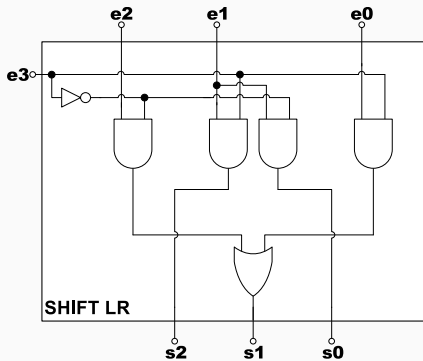


Glitches... oh my!

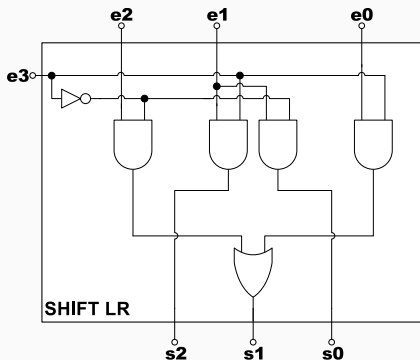
Glitches... oh my!



Revisitemos nuestro Shift LR:

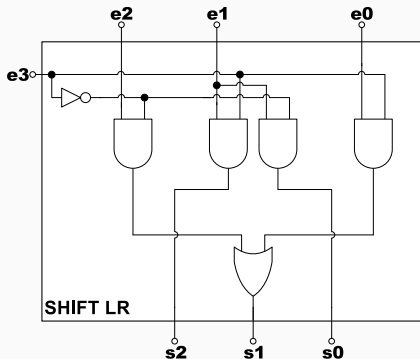


Revisitemos nuestro Shift LR:



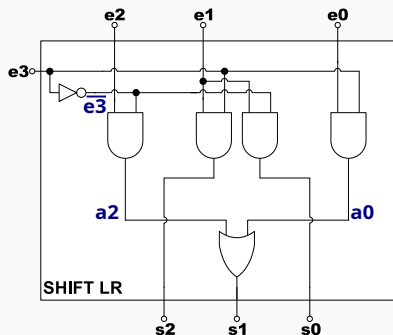
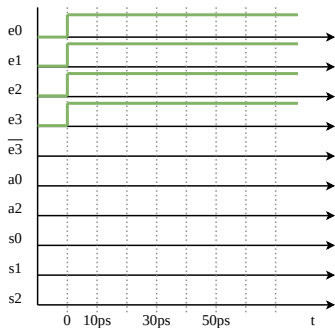
- ¿Cuál es camino crítico?

Revisitemos nuestro Shift LR:



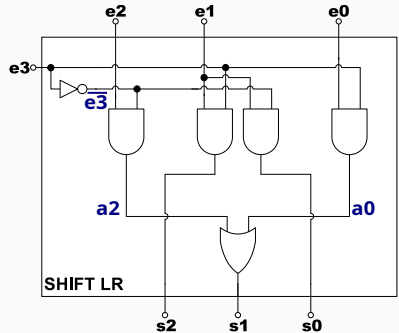
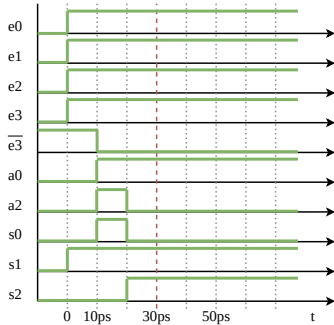
- ¿Cuál es camino crítico?
- ¿Hay glitches?

Hagamos un diagrama de tiempos³ suponiendo que todas las compuertas tardan 10 ps en poner un resultado válido en sus salidas:

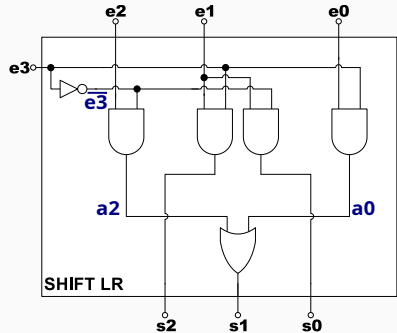
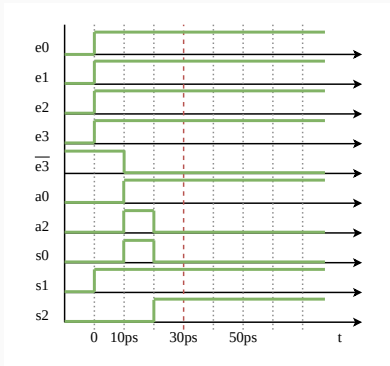


³Y nombremos a las señales que no tienen nombre

Nos queda:

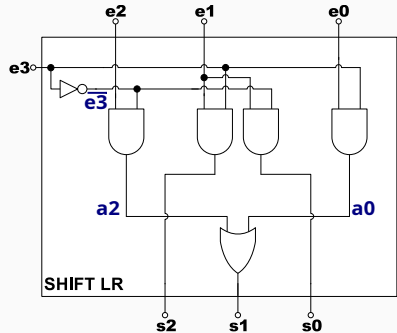
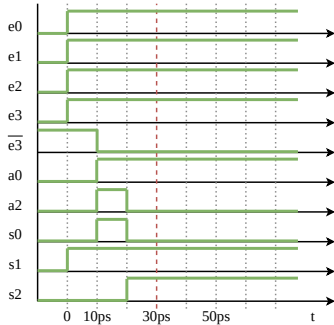


Nos queda:



- Camino crítico: $\overline{e3} \rightarrow a2 \rightarrow s1$.

Nos queda:



- Camino crítico: $\overline{e3} \rightarrow a2 \rightarrow s1$.
- Tenemos glitches.

¿Cuál es el mínimo tiempo que se debe esperar para leer un resultado válido de su salida?

¿Cuál es el mínimo tiempo que se debe esperar para leer un resultado válido de su salida?

- En este caso debemos esperar al menos $3 \cdot 10ps = 30ps$ para poder leer la salida.

¿Cuál es el mínimo tiempo que se debe esperar para leer un resultado válido de su salida?

- En este caso debemos esperar al menos $3 \cdot 10ps = 30ps$ para poder leer la salida.

¿Cómo enfrentamos (en parte) este problema?
Secuenciales... (en el próximo episodio)

¿Preguntas?
